

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
ФАКУЛЬТЕТ АВТОМАТИКИ И ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ
КАФЕДРА ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

КУРСОВАЯ РАБОТА

по дисциплине "Технология разработки программного обеспечения"

**Прототип программной системы последовательной
идентификации временных рядов**

Студент: Щербаченко А.А.

Группа: АПИМ-13

Преподаватель:

к.т.н.,

доцент каф. ВТ

Романов Е.Л.

Новосибирск, 2014

СОДЕРЖАНИЕ

1	Общее видение программного проекта	3
2	Модель предметной области	4
3	Определение требований	4
3.1	Построение модели требований	4
3.2	Выявление актеров и прецедентов	6
3.3	Спецификации прецедентов	7
3.4	Модель прецедентов	17
4	Анализ	18
4.1	Модель классов анализа	18
5.1	Реализация прецедентов	19
4.3	Структура базы данных	19
5	Проектирование	21
6	Реализация	30
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	31

1 ОБЩЕЕ ВИДЕНИЕ ПРОГРАММНОГО ПРОЕКТА

На данный момент на кафедре ВТ НГТУ разработана и зарегистрирована программа VarForecasts, включающая комплекс стандартных, а также оригинальных алгоритмов прогнозирования временных рядов, что позволяет провести исследование и прогнозирование временных рядов на основе идей и методов вариативного моделирования. Программа реализована как настольное приложение с графическим интерфейсом, написанное с помощью технологии MFC (Microsoft Foundation Classes), поэтому его запуск возможен только под управлением ОС семейства Windows. Кроме того, существует проблема доставки обновления для пользователя: после внесения в программу дополнительной функциональности (к примеру, добавление нового прогнозного метода) потребуется доставить копию новой программы каждому из пользователей. По этим и другим причинам была поставлена задача перевода настольного приложения VarForecasts на клиент-серверную основу, учитывая современные тенденции в области разработки статистического программного обеспечения. На данном этапе новая версия VarForecasts обладает следующими возможностями:

- 1) хранение, преобразование и редактирование данных, представимых временными рядами, в наглядном виде;
- 2) идентификация и прогнозирование дискретных временных рядов разной дискретности на основе метода «Фурье-анализ»;
- 3) расширяемость системы за счет добавление других методов идентификации и прогнозирования временных рядов.

Текущей задачей является добавление в новую версию системы VarForecasts нового метода последовательной идентификации временных рядов. Программный модуль, реализующий новый метод, должен в диалоговом и автоматическом режимах обеспечивать возможность построения модели временного ряда путем поэтапного выделения его структурных компонент и их параметрического описания при помощи комплекса вероятностно-статистических методов (SSA, ARIMA и другие).

2 МОДЕЛЬ ПРЕДМЕТНОЙ ОБЛАСТИ

Каждый пользователь должен иметь свою учетную запись для работы с системой. Данные временных рядов хранятся в документах-таблицах, таблицы хранятся в директориях, при этом директории не могут содержать другие документы - только таблицы. Результат применения прогнозного метода над некоторым временным рядом оформляется в виде документа-отчета и связывается с таблицей. Директории, таблицы и отчеты образуют иерархическое дерево документов, которое должно всегда быть доступно пользователю. Клик ЛКМ по документу в дереве документов открывает его свойства/настройки. Дерево документов и страницы свойств/настроек документов расположены в рабочей области системы. Модель классов, представляющих описанную "предметную область", приведена на рисунке 1.

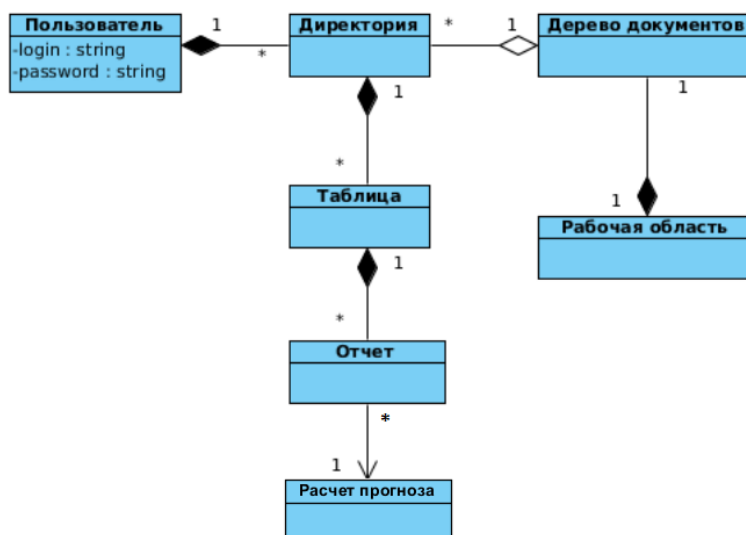


Рисунок 1 – Модель классов предметной области.

3 ОПРЕДЕЛЕНИЕ ТРЕБОВАНИЙ

3.1 Построение модели требований

Модель требований UP содержит формализованное описание функциональных и нефункциональных требований к ПС с указанием следующих приоритетов:

- M - Must have - обязательные к реализации требования;
- S - Should have - желательные требования;
- C - Could have - возможные требования;

- W - Want to have - перспективные требования, которые могут быть реализованы в следующих версиях ПС.

Функциональные требования:

- 1) аутентификация пользователей - M;
- 2) регистрация пользователей - M;
- 3) создание, удаление, переименование директорий - M;
- 4) создание, удаление, переименование таблиц - M;
- 5) редактирование данных таблиц - M;
- 6) возможность снабжения таблиц метаданными даты - M;
- 7) создание, удаление, переименование отчётов - M;
- 8) выбор прогнозного метода для создания отчета - M;
- 9) просмотр временного ряда, восстановленного по прогнозной модели - M;
- 10) сравнение исходного и модельного временных рядов - M;
- 11) идентификация и прогнозирование временных рядов на основе метода "Фурье-анализ" - M;
- 12) последовательная идентификация временных рядов (SSA-ARIMA):
 - a. построение графика исходного временного ряда - W;
 - b. вывод статистических характеристик исходного BP - M;
 - c. выбор режима работы алгоритма идентификации (диалоговый или автоматический) - M;
- 13) построение графика временного ряда - W.
- 14) хранение нескольких временных рядов в одной таблице - W.

Нефункциональные требования:

- по безопасности - использование HTTPS для аутентификации пользователей - C;
- по надежности - резервное копирование базы данных системы - S;
- по архитектуре:
 - клиент-серверная архитектура на базе технологий Java EE - M;
 - сервер приложений Apache Tomcat- M;
 - СУБД PostgreSQL- M;
 - AJAX для клиентской части – S;
 - возможность добавления в систему новых методов прогнозирования - M.

3.2 Выявление актеров и прецедентов

С системой взаимодействует только один актер в роли Пользователя. Для работы с системой необходима регистрация. Определения терминов и понятий, с которыми имеет дело Пользователь в процессе работы с системой, приведены в таблице 1.

Таблица 7. Глоссарий проекта

Термин	Определение
Рабочая область	Главная страница системы, разделенная на две части: в левой части располагается дерево документов, в правой - содержимое и/или настройки выбранного Пользователем документа. Синонимы: рабочее пространство.
Дерево документов	Отображение иерархической структуры документов в виде дерева, подобного "дереву файлов и каталогов" в файловых менеджерах. Обеспечивает возможность навигации по документам рабочей области. Синонимы: нет.
Документ	Элемент организации, структурирования и хранения данных. Приняты следующие типы документов: директория, таблица отчет. Синонимы: нет.
Директория	Аналог понятия директории, каталога или папки в файловых системах за тем исключением, что директории не поддерживают вложенность и могут содержать только таблицы. Синонимы: каталог, папка.
Таблица	Документ, содержащий исходный временной ряд в табличном виде; изначально содержит два столбца - номер элемента в ряду и его значение, может быть добавлен дополнительный столбец с календарной информацией. С таблицей могут быть связаны отчеты. Синонимы: нет.
Отчет	Документ, содержащий результаты построения прогнозной модели для некоторого временного ряда по выбранному методу прогнозирования. Синонимы: нет.

С Пользователем связаны следующие прецеденты:

- аутентификация;
- регистрация;
- создать директорию;
- переименовать директорию;
- удалить директорию;
- создать таблицу;
- создать таблицу с импортом данных;
- переименовать таблицу;
- удалить таблицу;
- вставить строку в таблицу;
- редактировать строку таблицы;
- удалить строки таблицы;
- сохранить таблицу;

- добавить метаданные в таблицу;
- создать отчет;
- переименовать отчет;
- удалить отчет;
- просмотреть модельный временной ряд.

3.3 Спецификации прецедентов

Прецедент: Регистрация

- Краткое описание: Система создает новую учетную запись для Пользователя;
- Актёры: Пользователь;
- Предусловия: нет;

Основной поток:

1. Прецедент начинается, когда Пользователь переходит на страницу создания новой учетной записи.
2. Система запрашивает у Пользователя данные для новой учетной записи (логин и пароль с подтверждением).
3. Система проверяет действительность данных Пользователя.
4. Система создает новую учетную запись для Пользователя.
5. Система сообщает Пользователю об успешной регистрации и переводит его на страницу для аутентификации.

Альтернативные потоки:

- 3.1. Данные недействительны - пользователь ввел логин, уже занятый другим пользователем:
 1. Система уведомляет Пользователя о занятом логине.
 2. Альтернативный поток возвращается на шаг 2 основного потока.
- 3.2. Данные недействительны - пароль не совпадает с подтверждением:
 1. Система уведомляет Пользователя, что введенные пароли не совпадают.
 2. Альтернативный поток возвращается на шаг 2 основного потока.
- Пользователь отменяет создание учетной записи.

Прецедент: Аутентификация

- Краткое описание: аутентификация нового Пользователя (вход Пользователя в систему);

- Актёры: Пользователь;
- Предусловия: Пользователь не прошел аутентификацию;

Основной поток:

1. Прецедент начинается, когда Пользователь запускает систему и открывается страница входа в систему.
2. Система запрашивает у Пользователя логин и пароль его учетной записи.
3. Система проверяет введенные данные.
4. Система начинает работу с учетной записью Пользователя и отображает страницу рабочей области.

Альтернативные потоки:

- 3.1. Данные недействительны - Пользователь ввел неправильный логин или пароль:
 1. Система уведомляет Пользователя, что логин и/или пароль неверны.
 2. Альтернативный поток возвращается на шаг 2 основного потока.

Прецедент: Создать директорию

- Краткое описание: создание новой директории в рабочей области Пользователя;
- Актёры: Пользователь;
- Предусловия: Пользователь прошел аутентификацию;

Основной поток:

1. Прецедент начинается, когда Пользователь нажимает кнопку создания новой директории.
2. Система запрашивает имя новой директории.
3. Система проверяет введенные данные.
4. Система создает новую директорию.
5. Система уведомляет Пользователя об успешном создании новой директории и выводит страницу управления директорией с обновленным деревом документов.

Альтернативные потоки:

- 3.1. Данные недействительны - директория с именем, введенным Пользователем, уже существует в рабочей области:
 1. Система уведомляет Пользователя о невозможности создать директорию с таким именем.
 2. Альтернативный поток возвращается на шаг 2 основного потока.
- Пользователь отменяет создание новой директории

Прецедент: Создать таблицу

- Краткое описание: создание новой таблицы в выбранной директории;
- Актёры: Пользователь;
- Предусловия: Пользователь прошел аутентификацию; выбрана директория, в которой будет создана таблица;

Основной поток:

1. Прецедент начинается, когда Пользователь нажимает кнопку создания таблицы.
2. Система запрашивает имя новой таблицы.
3. Система проверяет введенные данные.
4. Система создает новую таблицу.
5. Система уведомляет Пользователя об успешном создании новой таблицы и выводит страницу работы с таблицей с обновленным деревом документов.

Альтернативные потоки:

- 3.1. Данные недействительны - таблица с именем, введенным пользователем, уже существует в выбранной директории:
 1. Система уведомляет Пользователя о невозможности создать таблицу с таким именем.
 2. Альтернативный поток возвращается на шаг 2 основного потока.

Прецедент: Создать таблицу с импортом данных

- Краткое описание: создание новой таблицы в выбранной директории и заполнение её данными из файла, указанного Пользователем;
- Актёры: Пользователь;
- Предусловия: Пользователь прошел аутентификацию; выбрана директория, в которой будет создана таблица;

Основной поток:

1. Прецедент начинается, когда Пользователь нажимает кнопку создания таблицы.
2. Система запрашивает имя новой таблицы и файл для импорта данных.
3. Система проверяет введенные данные.
4. Система создает новую таблицу.
5. Система уведомляет Пользователя об успешном создании новой таблицы и выводит страницу работы с таблицей с обновленным деревом документов.

Альтернативные потоки:

- 3.1. Данные недействительны - таблица с именем, введенным пользователем, уже существует в выбранной директории:
 1. Система уведомляет Пользователя о невозможности создать таблицу с таким именем.

2. Альтернативный поток возвращается на шаг 2 основного потока.

- 3.2. Данные недействительны - файл для импорта не может быть прочитан или имеет неподходящий формат:

1. Система уведомляет Пользователя о невозможности создать таблицу и импортировать данные из указанного файла.

2. Альтернативный поток возвращается на шаг 2 основного потока.

Прецедент: Создать отчет

- Краткое описание: создание отчета по выбранному методу прогноза для выбранной таблицы;

- Актёры: Пользователь;

- Предусловия: Пользователь прошел аутентификацию; выбрана таблица, для которой будет создан отчет;

Основной поток:

1. Прецедент начинается, когда Пользователь выбирает опцию "создать новый отчет".

2. Пользователь вводит имя нового отчета, выбирает метод прогноза и вызывает мастер прогнозов.

3. Система проверяет введенные данные.

4. Система создает новый отчет.

5. Система уведомляет Пользователя об успешном создании нового отчета и выводит страницу настроек прогноза с обновленным деревом документов.

точка расширения: РасчетМодели

точка расширения: СравнениеВременныхРядов

Альтернативные потоки:

- 3.1. Данные недействительны - отчет с именем, введенным пользователем, уже связан с выбранной таблицей:

1. Система уведомляет Пользователя о невозможности создать отчет с таким именем.

2. Альтернативный поток возвращается на шаг 2 основного потока.

Расширяющий прецедент: Расчет по методу "Фурье-анализ"

- Краткое описание: Сегмент 1: расчет прогнозной модели по методу "Фурье-анализ" для выбранной таблицы;

- Актёры: Пользователь;

- Сегмент 1 предусловия: выбран отчет;

Поток сегмента 1:

1. Пользователь выбирает способ построения модели прогноза.
2. Система выполняет расчет модели в соответствии с заданными параметрами.
3. Система выводит результаты расчета модели.

Расширяющий прецедент: Расчет по методу "Последовательная идентификация"

- Краткое описание: Сегмент 1: расчет прогнозной модели по методу " Последовательная идентификация " для выбранной таблицы;
- Актёры: Пользователь;
- Сегмент 1 предусловия: выбран отчет;

Поток сегмента 1:

1. Пользователь выбирает режим работы алгоритма идентификации.
2. Если режим работы – «диалоговый».
 - 2.1. Пользователь выбирает параметры алгоритма SSA.
 - 2.2. Система выполняет расчет компонент (этап SSA «Разложение»).
 - 2.3. Пользователь выполняет классификацию компонент на тренд и сезонные составляющие BP.
 - 2.4. Система формирует модель BP путем его восстановления по выбранным компонентам (этап SSA «Восстановление»).
 - 2.5. Система отображает результаты идентификации – модель BP (тренд и сезонные составляющие), модельные остатки (график и статистические характеристики), проверяет адекватность модели BP.
 - 2.6. Если Пользователь принимает решение о проведении идентификации с другими параметрами.
 - 2.6.1 Переход на шаг 2.1.
 - 2.7. Иначе Если Пользователь принимает решение о продолжении идентификации.
 - 2.7.1. Пользователь выбирает метод для анализа модельных остатков и его параметры.
 - 2.7.2. Система выделяет очередную компоненту из остатков и добавляет её в модель BP.
 - 2.7.3. Система отображает результаты идентификации – модель BP с учетом новой составляющей, модельные остатки (график и статистические характеристики), проверяет адекватность модели BP.
 - 2.7.4. Переход на шаг 2.7.

2.8. Иначе

2.8.1 Система выводит окончательные результаты расчета модели.

3. Иначе

3.1. Система выполняет расчет модели в соответствии с заданными параметрами.

4. Система выводит окончательные результаты расчета модели.

Расширяющий прецедент: Сравнить временные ряды

- Краткое описание: Сегмент 1: вывод результатов сравнения исходного и модельного временных рядов;
- Актёры: Пользователь;
- Сегмент 1 предусловия: Пользователь прошел аутентификацию; выбран отчет; построена модель временного ряда по выбранному методу;

Поток сегмента 1:

1. Пользователь выбирает опцию "сравнение известных значений с моделью".
2. Система выводит результаты сравнения временных рядов.

Прецедент: Переименовать директорию

- Актёры: Пользователь;
- Предусловия: Пользователь прошел аутентификацию; выбрана директория;

Основной поток:

1. Прецедент начинается, когда Пользователь выбирает опцию "переименовать директорию".
2. Система запрашивает новое имя для директории.
3. Система проверяет введенные данные.
4. Система изменяет имя директории.
5. Система уведомляет Пользователя об успешном изменении имени директории и выводит страницу управления директорией с обновленным деревом документов.

Альтернативные потоки:

- 3.1. Данные недействительны - директория с именем, введенным пользователем, уже существует в рабочей области:
 1. Система уведомляет Пользователя о невозможности присвоить директории новое имя.
 2. Альтернативный поток возвращается на шаг 2 основного потока.

Прецедент: Переименовать таблицу

- Актёры: Пользователь;
- Предусловия: Пользователь прошел аутентификацию; выбрана таблица;

Основной поток:

1. Прецедент начинается, когда Пользователь выбирает опцию "переименовать таблицу".
2. Система запрашивает новое имя для таблицы.
3. Система проверяет введенные данные.
4. Система изменяет имя таблицы.
5. Система уведомляет Пользователя об успешном изменении имени таблицы и выводит страницу работы с таблицей с обновленным деревом документов.

Альтернативные потоки:

- 3.1. Данные недействительны - таблица с именем, введенным пользователем, уже существует в текущей директории:
 1. Система уведомляет Пользователя о невозможности присвоить таблице новое имя.
 2. Альтернативный поток возвращается на шаг 2 основного потока.

Прецедент: Переименовать отчет

- Актёры: Пользователь;
- Предусловия: Пользователь прошел аутентификацию; выбран отчет;

Основной поток:

1. Прецедент начинается, когда Пользователь выбирает опцию "переименовать отчет".
2. Система запрашивает новое имя для отчета.
3. Система проверяет введенные данные.
4. Система изменяет имя отчета.
5. Система уведомляет Пользователя об успешном изменении имени отчета и выводит страницу настроек прогноза с обновленным деревом документов.

Альтернативные потоки:

- 3.1. Данные недействительны - отчет с именем, введенным пользователем, уже связан с выбранной таблицей:
 1. Система уведомляет Пользователя о невозможности присвоить директории новое имя.
 2. Альтернативный поток возвращается на шаг 2 основного потока.

Прецедент: Удалить директорию

- Краткое описание: удаление директории со всем содержимым (таблицами и отчетами);
- Актёры: Пользователь;

- Предусловия: Пользователь прошел аутентификацию; выбрана директория;

Основной поток:

1. Прецедент начинается, когда Пользователь выбирает опцию "удалить директорию".
2. Пользователь подтверждает удаление.
3. Система удаляет директорию и связанные с ней таблицы из БД.
4. Система уведомляет Пользователя об успешном удалении директории и выводит страницу рабочей области с обновленным деревом документов.

Альтернативные потоки:

- 2.1. Пользователь отменяет удаление:
 1. Система выводит страницу управления директорией.

Прецедент: Удалить таблицу

- Краткое описание: удаление таблицы со всем отчетами, связанными с ней;
- Актёры: Пользователь;
- Предусловия: Пользователь прошел аутентификацию; выбрана таблица;

Основной поток:

1. Прецедент начинается, когда Пользователь выбирает опцию "удаление таблицы".
2. Пользователь подтверждает удаление.
3. Система удаляет таблицу и связанные с ней отчеты из БД.
4. Система уведомляет Пользователя об успешном удалении таблицы и выводит страницу рабочей области с обновленным деревом документов.

Альтернативные потоки:

- 2.1. Пользователь отменяет удаление:
 1. Система выводит страницу работы с таблицей.

Прецедент: Удалить отчет

- Актёры: Пользователь;
- Предусловия: Пользователь прошел аутентификацию; выбран отчет;

Основной поток:

1. Прецедент начинается, когда Пользователь выбирает опцию "удаление отчета".
2. Пользователь подтверждает удаление.
3. Система удаляет отчет из БД.
4. Система уведомляет Пользователя об успешном удалении отчета и выводит страницу рабочей области с обновленным деревом документов.

Альтернативные потоки:

- 2.1. Пользователь отменяет запрос:

1. Система выводит страницу настроек прогноза.

Прецедент: Вставить строку

- Краткое описание: вставка новой строки (с пустыми ячейками) в любую позицию выбранной таблицы.

- Актёры: Пользователь;

- Предусловия: Пользователь прошел аутентификацию; выбрана таблица;

Основной поток:

1. Прецедент начинается, когда Пользователь выбирает опцию "вставить строку" и позицию для вставки.

2. Система вставляет строку с пустыми ячейками в таблицу.

Альтернативные потоки: нет.

Прецедент: Редактировать строку

- Краткое описание: изменение значений ячеек выбранной строки таблицы.

- Актёры: Пользователь;

- Предусловия: Пользователь прошел аутентификацию; выбрана таблица; выбрана ячейка таблицы.

Основной поток:

1. Прецедент начинается, когда Пользователь выделяет ячейку таблицы (посредством нажатия ЛКМ или переходом по клавише TAB).

2. Пользователь вводит новое значение в ячейку таблицы.

3. Система проверяет введенное значение.

4. Система отображает введенное значение.

Альтернативные потоки:

- 3.1. Данные недействительны - Пользователь ввел нечисловое значение в ячейку таблицы:

1. Система уведомляет Пользователя о недопустимости нового значения ячейки таблицы.

2. Система отображает исходное не измененное значение ячейки таблицы.

3. Альтернативный поток возвращается на шаг 2 основного потока.

Прецедент: Удалить строки

- Краткое описание: удаление выбранных строк таблицы.

- Актёры: Пользователь;
- Предусловия: Пользователь прошел аутентификацию; выбрана таблица;

Основной поток:

1. Прецедент начинается, когда Пользователь выбирает опцию "удалить строку".
2. Система удаляет строку из таблицы.

Альтернативные потоки:

- Удаление нескольких строк:
 1. Прецедент начинается, когда Пользователь выбирает опцию "удалить строки".
 2. Пользователь вводит число строк для удаления и указывает номер строки, с которой следует начать удаление.
 3. Система удаляет строки из таблицы.

Прецедент: Добавить метаданные

- Краткое описание: добавление дополнительного столбца в таблицу, содержащего календарную информацию - дату добавления значения в временной ряд.

- Актёры: Пользователь;
- Предусловия: Пользователь прошел аутентификацию; выбрана таблица;

Основной поток:

1. Прецедент начинается, когда Пользователь выбирает опцию "добавить метаданные".
2. Пользователь выбирает частоту добавления значений в временной ряд и дату начала измерений.
3. Система добавляет в таблицу дополнительный столбец с календарными датами.

Альтернативные потоки: нет.

Прецедент: Сохранить таблицу

- Краткое описание: сохранение всех изменений, внесенных в таблицу посредством вставки, редактирования и удаления строк.

- Актёры: Пользователь;
- Предусловия: Пользователь прошел аутентификацию; выбрана таблица;

Основной поток:

1. Прецедент начинается, когда Пользователь выбирает опцию "сохранить таблицу".
2. Система сохраняет изменения, внесенные Пользователем в таблицу.
3. Система уведомляет Пользователя об успешном сохранении таблицы.

Альтернативные потоки: нет.

3.4 Модель прецедентов

Контекстная диаграмма основных функций ПС представлена на рисунке 3. Прецеденты "Создать документ", "Удалить документ", "Переименовать документ" и "Редактировать таблицу" являются абстрактными и введены для визуального уменьшения количество связей прецедентов с единственным актером. Прецеденты "Расчет по методу Фурье-анализ" и "Расчет по методу ССА" являются условными расширяющими прецедентами.

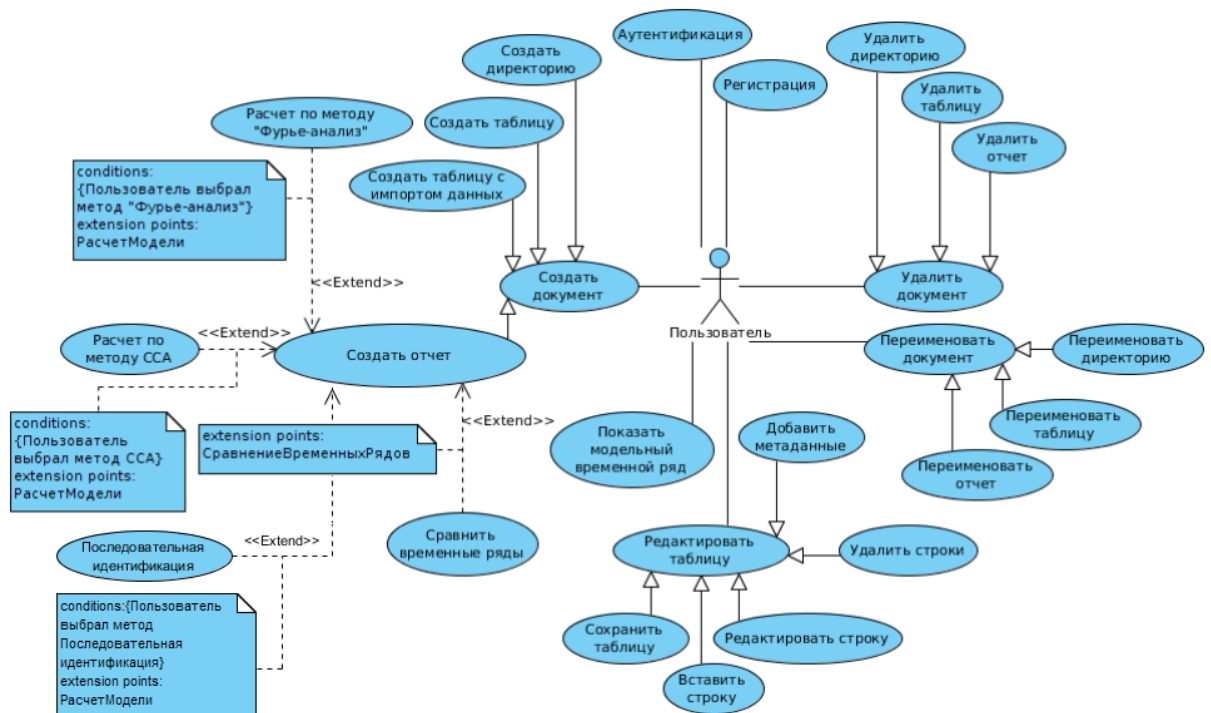


Рисунок 3 - Контекстная диаграмма прецедентов проектируемой ПС.

4 АНАЛИЗ

4.1 Модель классов анализа

Для выявления классов анализа использован метод стереотипов RUP. Идея этого метода состоит в том, что в ходе анализа рассматриваются три разных типа класса анализа: граничный класс, управляющий класс и класс сущности. Этот метод позволяет сосредоточить анализ на конкретных аспектах системы. Диаграмма классов анализа проектируемой ПС, построенная с использованием метода стереотипов RUP, приведена на рисунке 4.

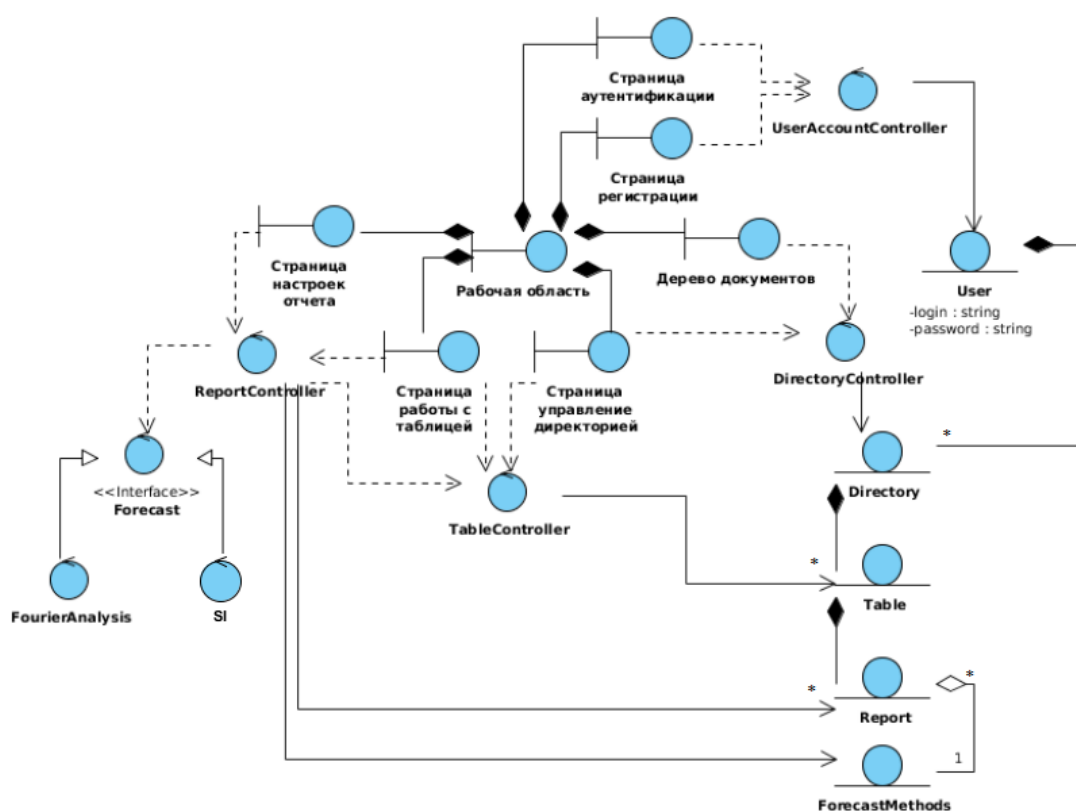


Рисунок 4 - Диаграмма классов анализа проектируемой ПС.

Назначение классов анализа:

- граничный класс – используется для моделирования взаимодействия между системой и актерами (пользователями, внешними системами или устройствами). Каждый граничный класс должен быть связан как минимум с одним актером;
- управляющий класс – отвечает за координацию, взаимодействие и управление другими объектами, выполняет сложные вычисления, управляет безопасностью и т. п.
- класс сущности – используется для моделирования долгоживущей, нередко сохраняемой информации. Классы сущности являются абстракциями основных понятий

предметной области – людей, объектов, документов и т. д., как правило, хранимых в табличном или ином виде.

4.2 Реализация прецедентов

После нахождения классов анализа ключевым моментом в анализе является поиск реализаций прецедентов. Они включают наборы классов, реализующих поведение, определенное в прецеденте – т.е. реализации прецедентов показывают, как взаимодействуют классы, чтобы реализовать функциональность системы. На рисунке 6 приведен фрагмент реализации прецедента с демонстрацией взаимодействия Пользователя с модулем идентификации (режим автоматической идентификации).

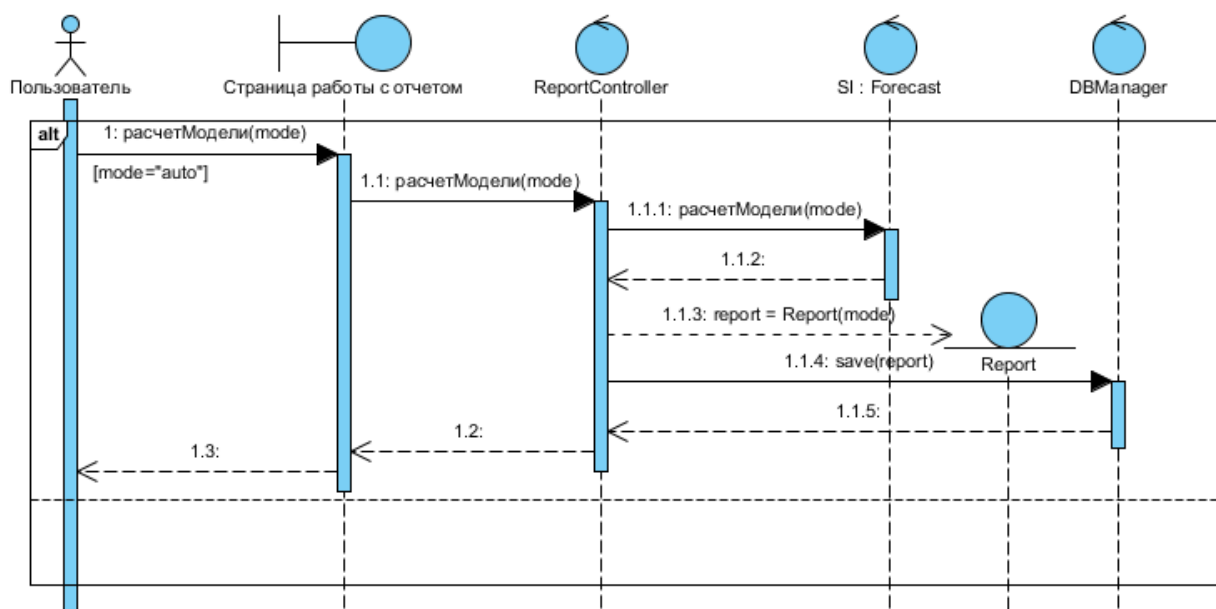


Рисунок 6 – Фрагмент реализации прецедента «Расчет по методу «Последовательная идентификация».

4.3 Структура базы данных

В диаграмме классов анализа классы-сущности представляют концептуальную модель БД проектируемой системы. На рисунке 5 приведена соответствующая ER-модель. Отношение ForecastMethods применяется в качестве списка прогнозных методов; поле class этого отношения обязано содержать имя присутствующего в программе класса, реализующего интерфейс Forecast. Остальные отношения моделируют иерархию

"пользователь-директории его рабочего пространства-таблицы с данными-прогнозы по данным таблиц".

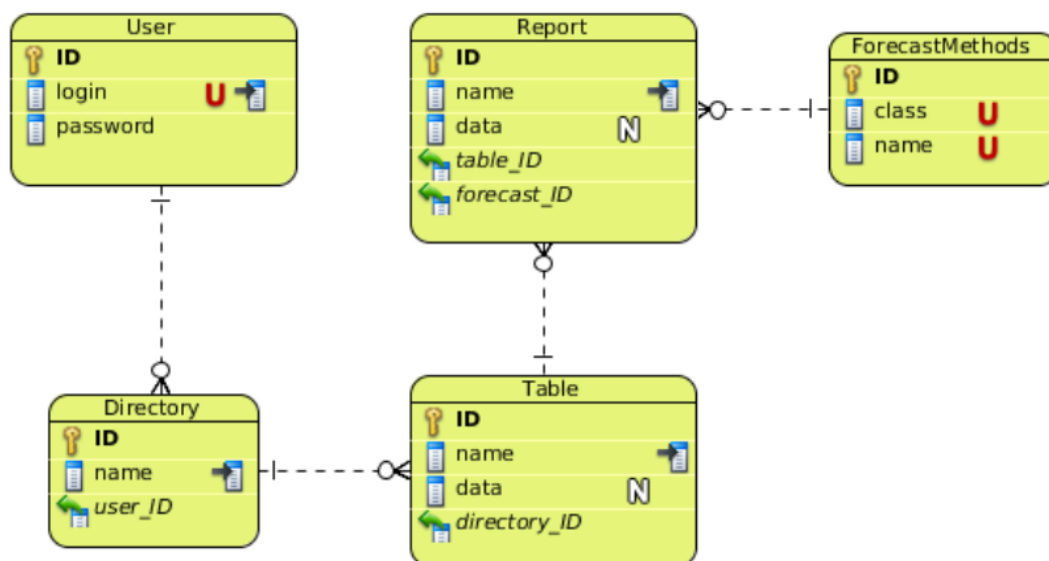


Рисунок 5 - Структура базы данных проектируемой ПС.

5 ПРОЕКТИРОВАНИЕ

Поскольку в качестве клиента в данном приложении используется веб-браузер, при написании программы учтены типичные ситуации, возникающие в работе веб-приложений. Так, при проектировании применён шаблон Post/Redirect/Get, в соответствии с которым при отправке браузером заполненной формы на веб-сервер последний, изменив данные по требованию пользователя, не сразу же возвращает ответ в виде HTML-страницы, а отправляет браузеру сообщение о необходимости обратиться к тому же серверу с другим URL-путём с Get-запросом. Страницу, полученную в результате данных действий, клиент может поместить в закладки браузера и обновить, при этом повторная отправка данных на сервер не произойдет.

В проекте пользователь работает со следующими видами адресов:

<Адрес сервера>

(Пользователь видит только дерево документов)

<Адрес сервера>/showDocument/<Имя открытой директории>

<Адрес сервера>/showDocument/<Имя открытой директории>/<Имя таблицы>

<Адрес сервера>/showDocument/<Имя открытой директории>/<Имя таблицы>/<Имя отчёта>

Следует заметить, что меню директории, страница с таблицей или отчёт по прогнозу демонстрируется только авторизованному пользователю, в противном случае отображается страница авторизации.

Также в программе используется популярный шаблон проектирования «Модель-представление-контроллер». Моделью является база данных, с которой работает сервер. Иерархия БД моделирует директории, таблицы и отчёты, находящиеся в пространстве пользователя, также в БД сохранены имена классов, которые позволяют проводить прогнозирование. За представление в программе отвечают jsp-страницы. На страницах отсутствуют какие-либо сложные вычисления, препятствующие пониманию размещённого на них HTML-кода, тем не менее, присутствуют вызовы методов, отображающих на странице дерево документов, таблицы с данными, отчёты по прогнозам с использованием объектов, хранящихся в http-сессии. Сервлет класса FrontControllerServlet является контроллером. В ответ на большинство запросов он обращается к БД с запросом чтения

либо правки данных, затем помещает в http-сессию необходимые данные и выбирает необходимую jsp-страницу, которая и формирует ответ.

Данная система разрабатывается с применением реляционной БД, а значит, структура используемых программой данных жёстко задана, и её изменение может повлечь необходимость переписать всё приложение. С другой стороны, ни таблицы данных, ни сохраняемые отчёты о проведённых прогнозах не имеют чётко определённой структуры: так, формула, представляющая ряд Фурье, может иметь произвольное число пар коэффициентов гармоник А и В; другой документ, описывающий другой прогнозный метод, будет иметь полностью отличную структуру; временной ряд может иметь метаданные, описывающие календарные даты, а может не иметь. Создать удобную в обработке схему реляционной БД, которая бы позволила хранить все виды документов-отчётов и документов-данных непосредственно в ячейках таблиц БД, не представляется возможным.

В связи с этим такие объекты программы, как таблицы и отчёты, представляются в виде строки формата JSON (Java Script Object Notation), хранящейся в одной ячейке реляционной БД. Данный текстовый формат позволяет описать абсолютно любую структуру данных, причём с использованием небольшого количества символов разметки.

Так как язык программирования Java изначально не имеет средств для работы с форматом JSON, для него написан специальный класс, позволяющий по JSON-строкам воссоздавать карты отображения (объекты класса Map), известные также как ассоциативные массивы, и проводить обратное преобразование.

При десериализации строки класс JsonSeDes проводит синтаксический анализ исходной строки методом рекурсивного спуска, и в процессе работы создаёт карту отображения, заполненную строками, числами, другими картами отображений и списками (класс List), заполненными данными тех же типов.

В конечном итоге применение формата JSON вполне оправдано, поскольку при извлечении записи со строкой формата JSON из базы данных обычно требуется считать все поля получаемой карты отображения или большинство.

Языкнезависимость данного формата также используется при обмене данными между серверной и клиентской частями программы, написанными на разных языках программирования. Применение формата JSON в этом контексте описывается ниже.

В ряде участков проекта потребовалось использовать возможности изменения веб-страницы без полной перезагрузки путём манипулирования объектной моделью документа,

заложенные в язык JavaScript. Следующие возможности программы основаны на модификации документа с помощью JavaScript:

- удаление, добавление строк таблицы с временным рядом;
- добавление новых строк в таблицу, снабжённую метаданными времени;
- отчёт о сравнении значений ряда и значений, выдаваемых моделью, встраиваемый в конец страницы отчёта по проведённому прогнозу по требованию пользователя;
- сохранение текущей копии отредактированной таблицы с рядом на сервере, отчёт о завершении сохранения.

При этом во всех этих случаях, кроме первого, веб-обозреватель обменивается с сервером данными в фоновом режиме; страница меняется лишь частично (этот подход известен как AJAX - «асинхронный JavaScript и XML», хотя данные могут передаваться в отличном от XML формате).

При обмене данными клиентская сторона отправляет в качестве запроса (а серверная в качестве ответа) строку в формате JSON.

Применение частичной перегрузки страницы делает интерфейс более удобным: так, пользователю не требуется обновлять страницу каждый раз, когда он добавляет строку к таблице. Также AJAX уменьшает объём трафика между клиентом и сервером: JSON-строки менее избыточны, чем HTML-документ ввиду меньшего числа разметочных символов и отсутствия фрагментов, описывающих части страницы, которые не подвергаются изменению. Наконец, экономия трафика происходит при вызове пользователем команды сохранения таблицы: в запрос попадают только строки, подвергшиеся изменению или отсутствовавшие в изначальной таблице, в то время как помещение редактируемых полей в составе таблицы внутрь html-формы потребовало бы отослать на сервер значение всех ячеек.

При включении в проект нового прогнозного метода программисту требуется создать класс-наследник интерфейса Forecast.

Проектировщику нового прогнозного метода, который должен быть встроен в систему, требуется реализовать определённые абстрактные методы.

Методы, относящиеся к представлению:

- вывода мастера прогнозов в формате HTML для объекта-прогноза, рассчитанного или неинициализированного;
- вывода отчёта в формате HTML по готовому расчёту;
- вывода строки с именем прогнозного метода.

Методы, относящиеся к модели:

- расчёта модели по поступившему в неё временному ряду и списку опций, полученных от заполненной пользователем формы мастера прогнозов;
- сохранения текущего состояния класса прогнозного метода в поля класса карты отображения Map;
- восстановления состояния класса прогнозного метода из полей класса карты отображения;
- расчёта значений ряда по готовой модели для заданных начального и конечного значений индекса.

После внесения данных о новом классе прогноза (имя класса, номер в списке прогнозов) в базу данных правильно написанный прогнозный метод становится полностью функциональным.

Таким образом, проектировщик прогнозного метода избавлен от необходимости изучать части программы, связанные с представлением данных, механизмами авторизации, и прочие, не относящиеся непосредственно к прогнозированию.

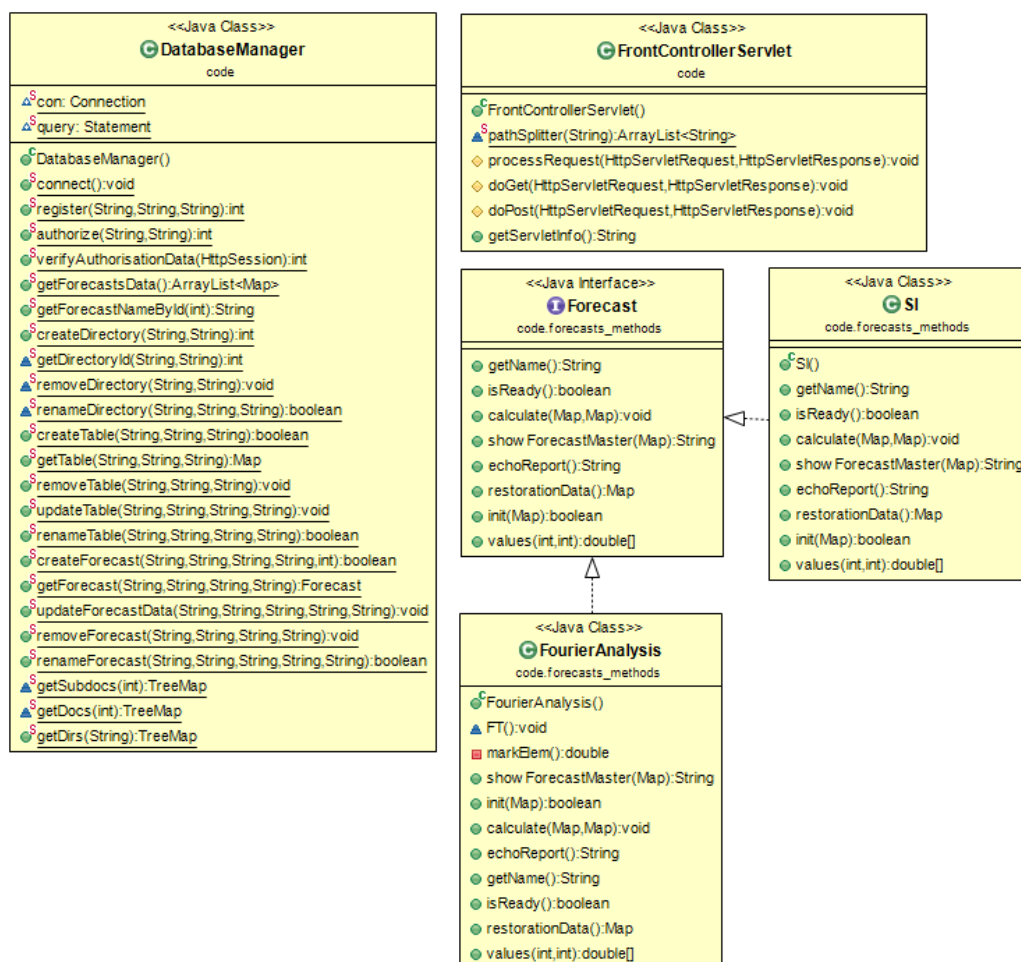


Рисунок 7 – Диаграмма проектных классов.

Класс **FrontControllerServlet**

Основной класс приложения, непосредственно взаимодействует с пользователем.

static ArrayList<String> pathSplitter(String path)

Вспомогательный метод, позволяющий разделить URL-путь запроса с целью определения, какой документ из рабочего пространства запросил пользователь.

protected void processRequest(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException

Метод, определяющий поведение сервлета при приёме запросов. В результате его работы браузер клиента получает готовую html-страницу (сгенерированную jsp-страницей, которой сервлет делегировал эту работу) либо сообщение о том, по какому URL-пути нужно сделать повторное обращение к веб-серверу.

protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException

protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException

Методы, отвечающие за приём сервлетом запросов. Делегируют всю работу методу processRequest.

public String getServletInfo()

Метод, возвращающий строку с описанием сервлета.

Класс **DatabaseManager**

Отвечает за все взаимодействия с БД.

Описание функций работы с БД, которые вызываются методами данного класса, приведено в Приложении Г.

public static void connect() throws SQLException, ClassNotFoundException

Инициализация соединения с БД. Исключение может возникнуть по причине невозможности установить соединение с сервером БД.

static public int register(String name, String pass, String pass2)

Регистрация нового пользователя.

```
static public int authorize(String name, String pass)
```

Авторизация пользователя.

```
static public int verifyAuthorisationData(HttpSession session)
```

Проверка ранее введенных авторизационных данных пользователя.

```
static public ArrayList<Map> getForecastsData()
```

Общий список доступных методов прогнозирования.

```
public static String getForecastNameById(int id)
```

Получение имени класса по номеру прогноза.

```
static public int createDirectory(String userName, String folderName)
```

Создание директории. Завершится с ошибкой, если имя директории занято другой директорией.

```
static int getDirectoryId(String userName, String folderName) throws Exception
```

Получение номера директории. Исключение возникает, если директории нет. (Метод применяется при необходимости выяснить, имеет ли пользователь в своём распоряжении директорию с таким номером).

```
static void removeDirectory(String userName, String folderName)
```

Удаление директории, если она существует.

```
static boolean renameDirectory(String userName, String oldFolderName, String newFolderName)
```

Переименование директории. Вернёт false, если имя директории занято другой директорией или директория отсутствует.

```
static public boolean createTable(String userName, String folderName, String tableName)
```

Создание пустой таблицы в некоторой директории. Вернёт false, если имя таблицы занято другой таблицей, или если связанная директория не существует.

static public Map getTable(String userName, String folderName, String tableName)
throws Exception

Извлечение данных таблицы из директории в виде карты отображения. Завершится с ошибкой, если таблица отсутствует либо данные о таблице записаны некорректно.

public static void removeTable(String userName, String folderName, String tableName)

Удаление таблицы, если она существует.

static public void updateTable(String userName, String folderName, String tableName, String data)

Обновление данных о таблице. Не изменит никаких данных, если таблица отсутствует.

public static boolean renameTable(String userName, String folderName, String oldTableName, String newTableName)

Переименование таблицы. Возврат false, если имя таблицы занято другой таблицей или таблица отсутствует.

static public boolean createForecast(String userName, String folderName, String tableName, String forecastName, int methodId)

Создание документа, содержащего отчёт по прогнозному методу. Вернёт false, если имя документа занято другим документом, или если связанная директория или таблица не существует.

static public Forecast getForecast(String userName, String folderName, String tableName, String forecastName) throws Exception

Извлечение отчёта по прогнозу из базы данных. Завершится с ошибкой, если документ отсутствует либо данные в БД записаны некорректно.

static public void updateForecastData(String userName, String folderName, String tableName, String forecastName, String data)

Обновление данных, описывающих отчёт, для его последующего восстановления. Не изменит никаких данных, если отчёт отсутствует.

static public void removeForecast(String userName, String folderName, String tableName, String forecastName)

Удаление отчёта, если он существует.

static public boolean renameForecast(String userName, String folderName, String tableName, String oldForecastName, String newForecastname)

Переименование отчёта. Вернёт false, если имя отчёта занято другим отчётом или отчёт отсутствует.

static TreeMap getSubdocs(int docid)

static TreeMap getDocs(int dirid)

public static TreeMap getDirs(String username)

Методы извлечения имён всех отчётов таблицы, всех таблиц директории и всех директорий пользователя соответственно.

Интерфейс Forecast

Интерфейс прогнозного метода, определяет как необходимые расчётные методы, так и методы ввода-вывода.

Любой класс, имплементирующий этот интерфейс и зарегистрированный в БД, должен иметь открытый конструктор по умолчанию.

String getName()

Возврат имени прогнозного метода для отображения.

boolean isReady()

Возврат признака готовности результата. Возвращается false, если модель не построена и отобразить отчёт невозможно.

void calculate(Map table, Map parameters)

Расчёт модели. Вход - документ с рядом в виде карты и карта настроек представления (ключи карты - имена атрибутов, указанные в мастере прогнозов как имена параметров запроса).

String showForecastMaster(Map data)

Метод отображения мастера прогнозов. На вход получает таблицу с данными, чтобы иметь возможность отображать мастера с разными предустановленными параметрами для разных таблиц данных.

String echoReport()

Вывод готового отчёта на страницу в формате HTML.

Map restorationData()

Перенос готового отчёта в карту отображения с целью последующего восстановления.

boolean init(Map data)

Восстановление отчёта из карты отображения, полученной из БД.

double[] values(int i, int i1)

Расчёт значений модели для некоторых идущих подряд индексов.

Класс SSA

Является оберткой над C++-классом, реализующим метод SSA. Для доступа к методам C++-класса используется интерфейс JNI.

void computeComponents(Object data, int L)

Вычисление всех компонент для заданного ряда с заданным лагом.

void performRestoring(float W0, float C0, float S0, float R0, Object Trend, Object Season)

Выполнить автоматическую классификацию компонент и восстановление трендовой и сезонной составляющей исходного временного ряда.

int getComponentsCount()

Получить количество рассчитанных компонент.

6 РЕАЛИЗАЦИЯ

В качестве клиента выступает веб-браузер (рисунок 8), расчеты моделей и прогнозирование временных рядов выполняются на веб-сервере Apache Tomcat. Для моделирования иерархии директорий и хранения данных о пользователях и прогнозных методах используется реляционная БД (СУБД – PostgreSQL).

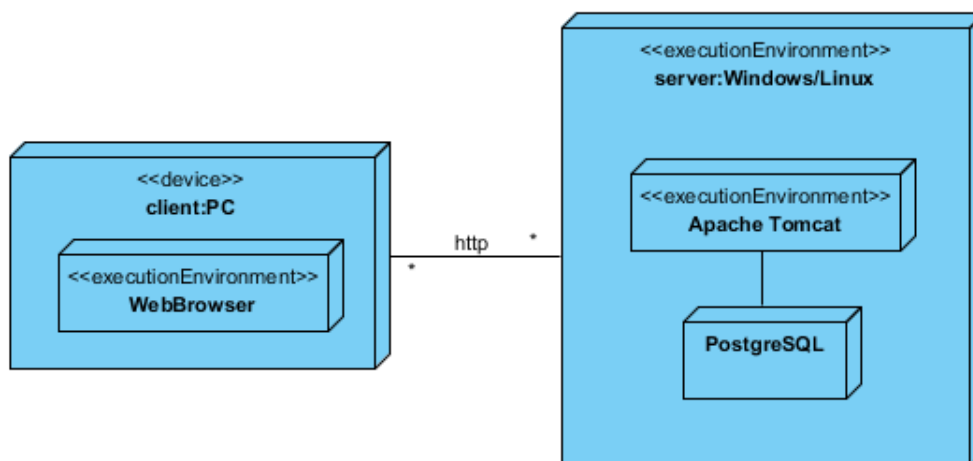


Рисунок 8 – Диаграмма развертывания.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Арлоу Д., Нейштадт И. UML 2 и Унифицированный процесс. Практический объектно-ориентированный анализ и проектирование, 2-е издание. – Пер. с англ. – СПб: СимволПлюс, 2007. – 624 с., ил.
2. Мартин Фаулер. UML. Основы. Краткое руководство по стандартному языку объектного моделирования. 3-е издание. Издательство “Символ-Плюс” (ISBN 5-93286-060-X), Санкт-Петербург, 2004.